

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент

должность, уч. степень,
звание

подпись, дата

Н.В. Богословская

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

по курсу: Большие данные

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Изучение методов построения и отбора признаков с применением встроенных компонентов Loginom и узла Python.

2 Задание

1. Реализовать рассмотренные в лабораторной работе этапы построения и отбора признаков для наборов данных как с использованием Python в соответствующем узле платформы Loginom, так и с помощью визуальных инструментов платформы.
2. Сформулировать аргументированные заключения о преимуществах или недостатках различных методов анализа.

3 Ход выполнения задания

В ходе работы был реализован метод отбора признаков и расчёт коэффициента корреляции Пирсона с помощью узла Python (скрипт `feature_engineering_gold.py`) и с помощью визуальных инструментов Loginom на наборе данных `GoldUSD.csv`. Общая схема сценария представлена на рисунке 1.

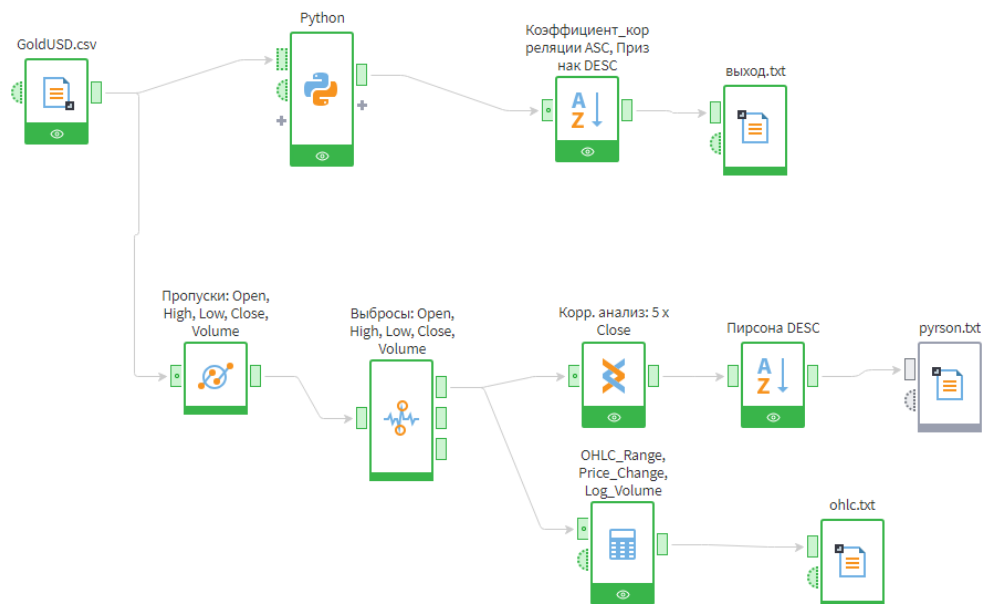


Рисунок 1 – Разработанная схема сценария

Настройки узла заполнения пропусков представлены на рисунке 2. Для числовых столбцов (`Open`, `High`, `Low`, `Close`, `Volume`) выбран метод заполнения медианой. Для даты использовано сохранение порядка (исходные данные упорядочены).

Заполнение пропусков

Исходные данные упорядочены ☒

Допустимый процент пропусков

Random seed

№	Входные поля Фильтрация	Вид данных	Метод обработки
1	31 Date	☒ Непрерывный	☐ Не выбран
2	9.0 Open	☒ Непрерывный	☒ Заменять медианой
3	9.0 High	☒ Непрерывный	☒ Заменять медианой
4	9.0 Low	☒ Непрерывный	☒ Заменять медианой
5	9.0 Close	☒ Непрерывный	☒ Заменять медианой
6	12 Volume	☒ Непрерывный	☒ Заменять медианой

Рисунок 2 – Настройки узла заполнения пропусков

Настройки узла редактирования выбросов представлены на рисунке 3. Обработка выбросов выполнена для числовых полей с заданием допустимого стандартного отклонения.

Редактирование выбросов

Исходные данные упорядочены ☒

☐	Входные поля Фильтрация	Вид данных
☐	31 Date	☒ Непрерывный
☒	9.0 Open	☒ Непрерывный
☒	9.0 High	☒ Непрерывный
☒	9.0 Low	☒ Непрерывный
☒	9.0 Close	☒ Непрерывный
☒	12 Volume	☒ Непрерывный

Рисунок 3 – Настройки узла редактирования выбросов

В узле Калькулятор были созданы признаки OHLC_Range (High – Low), Price_Change (Close – Open) и Log_Volume (ln(Volume+1)). Настройки калькулятора представлены на рисунке 4.

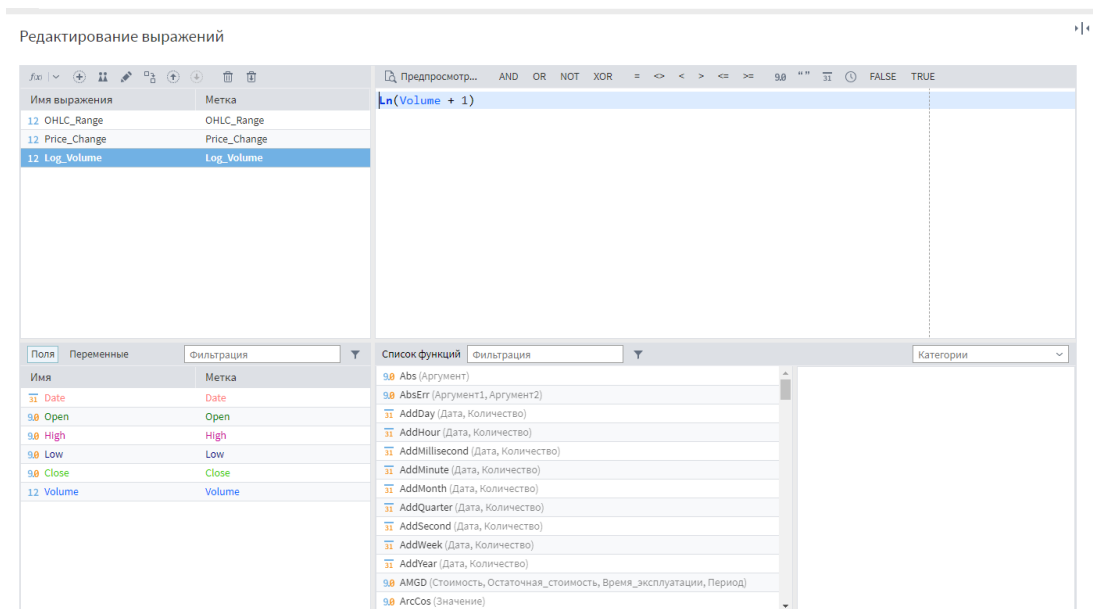


Рисунок 4 – Построение признаков в калькуляторе

Поиск корреляции проводился между признаками и целевой переменной Close. Настройки узла корреляционного анализа представлены на рисунке 5. Включён коэффициент корреляции Пирсона; целевая переменная Close выделена в отдельный набор (следствие).

Пирсона DESC • Быстрый просмотр						
Выходной набор данных						
#	9.0 Пирсона	ab Поле1.Имя	ab Поле1.Метка	ab Поле2.Имя	ab Поле2.Метка	
1	1,00	Low	Low	Close	Close	
2	1,00	High	High	Close	Close	
3	1,00	Open	Open	Close	Close	
4	0,86	Date	Date	Close	Close	
5	0,03	Volume	Volume	Close	Close	

Рисунок 5 – Настройки узла корреляционного анализа

Результат сортировки по Пирсону (визуальные инструменты Loginom) представлен на рисунке 6. Наибольшую корреляцию с Close имеют Low, High, Open ($\approx 0,9999$), затем Date ($\approx 0,86$) и Volume ($\approx 0,032$).

Рисунок 6 – Результат корреляционного анализа (узлы Loginom)

Результат отбора признаков с помощью узла Python представлен на рисунке 7. Скрипт выполняет заполнение пропусков, ограничение выбросов, построение признаков (OHLC_Range, Price_Change, MA5, MA20, Log_Volume, Year, Month, DayOfWeek и др.) и вычисление корреляций. Топ-признаки по модулю корреляции: High, Low, Open ($\approx 0,99$), Year ($\approx 0,86$), OHLC_Range ($\approx 0,63$), Log_Volume ($\approx 0,34$).

Кoeffициент корреляции ASC, Признак DESC		
Выходной набор данных		
#	ab Признак	9.0 Кoeffициент корреляции
1	Month	-0,01
2	DayOfWeek	0,00
3	Price_Change_Pct	0,03
4	Volume	0,03
5	Price_Change	0,07
6	Log_Volume	0,34
7	OHLC_Range	0,63
8	Year	0,86
9	MA20	0,99
10	MA5	0,99
11	Open	0,99
12	Low	0,99
13	High	0,99

Рисунок 7 – Результат отбора признаков (узел Python)

Результаты, полученные обоими способами, согласуются: цены Open, High, Low почти идеально коррелируют с Close (типично для OHLC-данных); Date и Year дают сильную связь (тренд цены золота); Volume — слабую. Небольшие отличия обусловлены разным составом признаков (Python добавляет MA5, MA20 и др.) и порядком обработки.

Что касается преимуществ и недостатков: узел Python позволяет точно настроить пайплайн и добавлять произвольные признаки; узлы Loginom удобнее настраивать визуально и не требуют знания программирования, но дают менее

гибкую настройку. Python даёт воспроизводимый скрипт; Loginom — прозрачный сценарий для быстрого прототипирования.

ПРИЛОЖЕНИЕ А

Листинг программы

```
import numpy as np
import pandas as pd
from typing import Optional, List, Tuple

# Попытка импорта модулей Loginom (доступны только при запуске в платформе)
try:
    import builtin_data
    from builtin_data import InputTable, OutputTable
    from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
    IN_LOGINOM = True
except ImportError:
    IN_LOGINOM = False

# =====
# Конфигурация
# =====
CSV_PATH = "GoldUSD.csv"
TARGET_COLUMN = "Close"
K_BEST = 20
DROP_NA_THRESHOLD = 0.5
OUTLIER_STD_THRESHOLD = 3.0
# =====

def load_data(df: Optional[pd.DataFrame] = None) -> pd.DataFrame:
    """Загрузка данных: из Loginom или из CSV."""
    if df is not None:
        return df.copy()
    if IN_LOGINOM:
        return to_data_frame(InputTable).copy()
    return pd.read_csv(CSV_PATH).copy()

def parse_date(df: pd.DataFrame) -> pd.DataFrame:
    """Парсинг даты и создание признаков времени."""
    df = df.copy()
    df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%y", errors="coerce")
    df["Year"] = df["Date"].dt.year
    df["Month"] = df["Date"].dt.month
    df["DayOfWeek"] = df["Date"].dt.dayofweek
    return df
```

```

def fill_missing(df: pd.DataFrame, numeric_cols: List[str]) -> pd.DataFrame:
    """Заполнение пропусков: числовые – медианой."""
    df = df.copy()
    for col in numeric_cols:
        if col in df.columns and df[col].isna().any():
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)
    return df

def drop_high_na_columns(df: pd.DataFrame, threshold: float = 0.5) -> pd.DataFrame:
    """Удаление столбцов с долей пропусков выше порога."""
    na_ratios = df.isna().mean()
    to_drop = [c for c, r in na_ratios.items() if r > threshold and c != TARGET_COLUMN]
    if to_drop:
        df = df.drop(columns=to_drop)
    return df

def handle_outliers(df: pd.DataFrame, numeric_cols: List[str],
                    n_std: float = 3.0) -> pd.DataFrame:
    """Ограничение выбросов (clip): не удаляем строки, а обрезаем экстремумы."""
    df = df.copy()
    for col in numeric_cols:
        if col not in df.columns or col == TARGET_COLUMN:
            continue
        mean_val = df[col].mean()
        std_val = df[col].std()
        if std_val > 0:
            lower = mean_val - n_std * std_val
            upper = mean_val + n_std * std_val
            df[col] = df[col].clip(lower=lower, upper=upper)
    return df

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """Построение признаков для OHLC-данных."""
    df = df.copy()
    if "High" in df.columns and "Low" in df.columns:
        df["OHLC_Range"] = df["High"] - df["Low"]
    if "Close" in df.columns and "Open" in df.columns:
        df["Price_Change"] = df["Close"] - df["Open"]
    if "Open" in df.columns and df["Open"].replace(0, np.nan).notna().any():

```

```
df["Price_Change_Pct"] = (df["Close"] - df["Open"]) / df["Open"].replace(0, np.nan)
```

```
# Скользящие средние (5 и 20 периодов)
```

```
if "Close" in df.columns:
```

```
    df["MA5"] = df["Close"].rolling(5, min_periods=1).mean()
```

```
    df["MA20"] = df["Close"].rolling(20, min_periods=1).mean()
```

```
# Объём: логарифм (Volume часто имеет много нулей)
```

```
if "Volume" in df.columns:
```

```
    df["Log_Volume"] = np.log1p(df["Volume"].fillna(0))
```

```
return df
```

```
def normalize_minmax(df: pd.DataFrame, cols: List[str]) -> pd.DataFrame:
```

```
    """Нормализация min-max: диапазон [0, 1]."""
```

```
    df = df.copy()
```

```
    for col in cols:
```

```
        if col not in df.columns:
```

```
            continue
```

```
        min_val = df[col].min()
```

```
        max_val = df[col].max()
```

```
        if max_val > min_val:
```

```
            df[col] = (df[col] - min_val) / (max_val - min_val)
```

```
    return df
```

```
def standardize(df: pd.DataFrame, cols: List[str]) -> pd.DataFrame:
```

```
    """Стандартизация: среднее=0, std=1."""
```

```
    df = df.copy()
```

```
    for col in cols:
```

```
        if col not in df.columns:
```

```
            continue
```

```
        mean_val = df[col].mean()
```

```
        std_val = df[col].std()
```

```
        if std_val > 0:
```

```
            df[col] = (df[col] - mean_val) / std_val
```

```
    return df
```

```
def correlation_table(X: pd.DataFrame, y: pd.Series) -> pd.DataFrame:
```

```
    """Таблица корреляций Пирсона: признак – коэффициент (отсортировано по |corr|)."""
```

```
    rows = []
```

```

y_std = (y - y.mean()).std(ddof=0)
if y_std == 0:
    return pd.DataFrame(columns=["Признак", "Коэффициент_корреляции"])

for col in X.columns:
    x = X[col]
    if x.std(ddof=0) == 0:
        continue
    try:
        corr = x.corr(y)
    except Exception:
        corr = np.nan
    if pd.notna(corr):
        rows.append((col, float(corr)))

corr_df = pd.DataFrame(rows, columns=["Признак", "Коэффициент_корреляции"])
corr_df["|corr|"] = corr_df["Коэффициент_корреляции"].abs()
corr_df = corr_df.sort_values("|corr|", ascending=False).reset_index(drop=True)
corr_df = corr_df.drop(columns=["|corr|"])
return corr_df

def select_top_features(corr_df: pd.DataFrame, k: int) -> List[str]:
    """Выбор топ-К признаков по модулю корреляции."""
    if corr_df.empty:
        return []
    return corr_df["Признак"].head(k).tolist()

def run(df_input: Optional[pd.DataFrame] = None) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """
    Основной пайплайн построения и отбора признаков.
    Возвращает: (processed_df, corr_table)
    """
    df = load_data(df_input)

    # 1. Парсинг даты
    if "Date" in df.columns:
        df = parse_date(df)

    # Числовые столбцы (исключая Date)
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    if TARGET_COLUMN not in df.columns:

```

```

        raise ValueError(f"Целевой столбец '{TARGET_COLUMN}' не найден в данных.")

# 2. Удаление столбцов с большим числом пропусков
df = drop_high_na_columns(df, DROP_NA_THRESHOLD)
numeric_cols = [c for c in numeric_cols if c in df.columns]

# 3. Заполнение пропусков
df = fill_missing(df, numeric_cols)

# 4. Построение признаков
df = engineer_features(df)

# Обновляем список числовых столбцов
feature_cols = [c for c in df.select_dtypes(include=[np.number]).columns
                 if c != TARGET_COLUMN]

# 5. Обработка выбросов (ограничение, не удаление)
df = handle_outliers(df, feature_cols, OUTLIER_STD_THRESHOLD)

# 6. Удаляем строки с NaN (после построения признаков могли появиться)
df = df.dropna(subset=[TARGET_COLUMN])

# 7. Матрица признаков и целевая переменная
X = df[feature_cols].copy()
y = df[TARGET_COLUMN]

# 8. Корреляционный анализ
corr_table = correlation_table(X, y)

# Top-K признаков
selected = select_top_features(corr_table, K_BEST)

print("=" * 60)
print("Лабораторная работа №3. Построение и отбор признаков")
print("=" * 60)
print(f"Строк: {len(df)}, Признаков: {len(feature_cols)}")
print(f"Топ-{K_BEST} признаков по корреляции с '{TARGET_COLUMN}':")
for i, row in corr_table.head(K_BEST).iterrows():
    print(f"  {row['Признак']}: {row['Коэффициент_корреляции']:.4f}")
print("=" * 60)

```

```

# Обработанный датасет (с Date и целевой переменной)
processed_df = df.copy()

return processed_df, corr_table

# =====
# Точка входа
# =====
if __name__ == "__main__":
    processed_df, corr_table = run()

    if IN_LOGINOM:
        # Вывод в Loginom: таблица корреляций
        if isinstance(OutputTable, type) or hasattr(OutputTable, '__class__'):
            prepare_compatible_table(OutputTable, corr_table, with_index=False)
            fill_table(OutputTable, corr_table, with_index=False)
    else:
        # Локальный запуск: сохранение результатов
        corr_table.to_csv("correlation_results.csv", index=False)
        print("Результаты сохранены в correlation_results.csv")

```